# ARIA: Data Pipeline & Project Architecture

# Explain the system clearly. Trace the data end to end. Defend the design decisions.

A repository-based guide to ARIA, the AI-powered personal finance assistant, financial tutor, market analyst and equity researcher.

## Start here if your interview is soon

• **10 minutes:** Read pages 2-3, the equity pipeline on page 9, and the answer bank on page 14.

• **30 minutes:** Add the shared pipeline, statement ingestion, model training and validation pages.

• **For a deep technical round:** Review all pages, then practise drawing the diagram from memory.

## Scope and evidence

Prepared from the current files in E:/ARIA. This guide describes source-code behavior; it does not assert that every optional model, provider, account integration or GPU service is installed or running. No live market requests, model training or full application tests were performed to prepare this document.

Older project reports describe a Streamlit prototype. The current application uses React + TypeScript and FastAPI. This guide prioritizes the current implementation. Proposed scaling work is explicitly marked as a future design.

**Use in the interview:** Replace ownership statements with the parts you personally built. Do not claim deployment scale, accuracy or latency that you have not measured.

# The project in 90 seconds

**Suggested explanation:** "ARIA brings four finance-related workflows into one conversational interface. A React frontend calls a FastAPI backend. Every question goes through a shared pipeline that normalizes the text, binds the user and conversation context, routes to a domain module, checks grounding and prepares the response for text or voice. Personal-finance calculations and equity valuations are computed in Python; language models mainly explain the results. SQLite stores user state and reports, while FAISS supports semantic concept retrieval. Optional voice and avatar services reuse the same answer path."

| Module | Core job | Main techniques |
|---|---|---|
| Personal finance | Profile-aware planning, spending, risk and budgeting | Deterministic engine; XGBoost; LSTM; Isolation Forest; LP |
| Financial tutor | Concept explanations, quizzes and adaptive teaching | Sentence-BERT + FAISS; LSTM knowledge tracing; DQN |
| Market analysis | Quotes, indices, instrument information and news | External market data; aggregation; LLM explanation |
| Equity research | Statements, ratios, valuation and saved research | Deterministic analysis; evidence ledger; narrative audit |

## The central design decision

Keep structured data and numerical computation separate from natural-language presentation. An LLM response is not the source of truth for a price, ratio, financial profile or valuation. Source coverage and guard strength vary by route; they do not provide a universal factual guarantee.

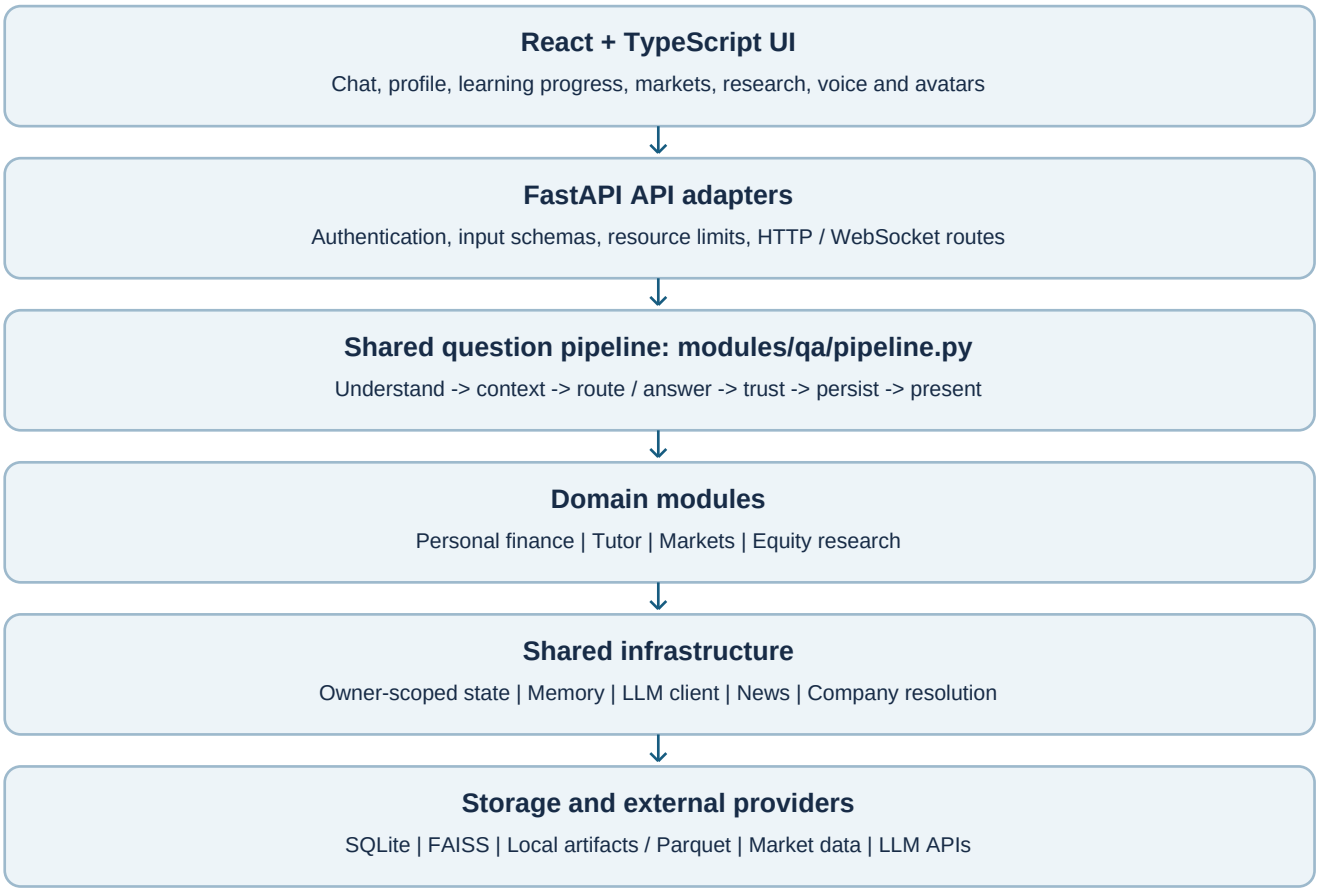
## Three pipelines to distinguish

• **Online serving:** a question becomes a contextual, grounded answer.

• **Data ingestion:** files, quotes, statements, concepts and headlines become validated structured records.

• **Offline learning and validation:** generated datasets or historical data produce model artifacts, backtests and measured evidence.

# System architecture




The arrows summarize dependencies. Data providers feed the domain modules; persistence is shared infrastructure rather than a mandatory final step of every provider call.

## Deployment topology

Development normally runs Vite on port 5173 and FastAPI on port 8000, with browser requests proxied to the API. The backend is a modular monolith: domain packages share one core application rather than independently deployed microservices. Optional avatar services run separately; forecast validation jobs can spawn isolated child processes.

**Why this shape?** It keeps a student-scale application easy to run and lets all channels reuse the same logic. The costs are shared process resources, tight deployment coupling and state that needs extra work before horizontal scaling.

**Sources:** api/main.py; web/package.json; modules/qa/pipeline.py; modules/equity_research/forecast/jobs.py.

# One question, end to end

| Stage | What happens | Relevant source |
|---|---|---|
| 1. Receive | The API creates a typed QARequest with query, owner, session, mode, channel, optional focus and request_id. | api/routes/chat.py; modules/qa/models.py |
| 2. Understand | Normalize spelling, text-speak and Hinglish. The orchestrator handles dialogue, domain and crisis-related behavior. | shared/understand.py; core/orchestrator.py |
| 3. Bind context | Set the current owner; verify conversation ownership; resolve focused instruments; attach memory and profile context. | modules/qa/pipeline.py; modules/qa/focus.py |
| 4. Execute a turn | Archive the request, lock the session and replay a completed retry when request_id matches. | core/session.py; shared/conversation_memory.py |
| 5. Route and compute | Resolve intent, company and page focus; run the relevant domain pipeline. | core/router.py; core/orchestrator.py |
| 6. Check trust | Apply route-specific grounding checks, disclosures and degraded-response handling. | modules/qa/trust.py |
| 7. Present and save | Prepare spoken/audio presentation where needed; complete the turn and checkpoint state. | modules/qa/present.py; core/session.py |

## Follow-up example

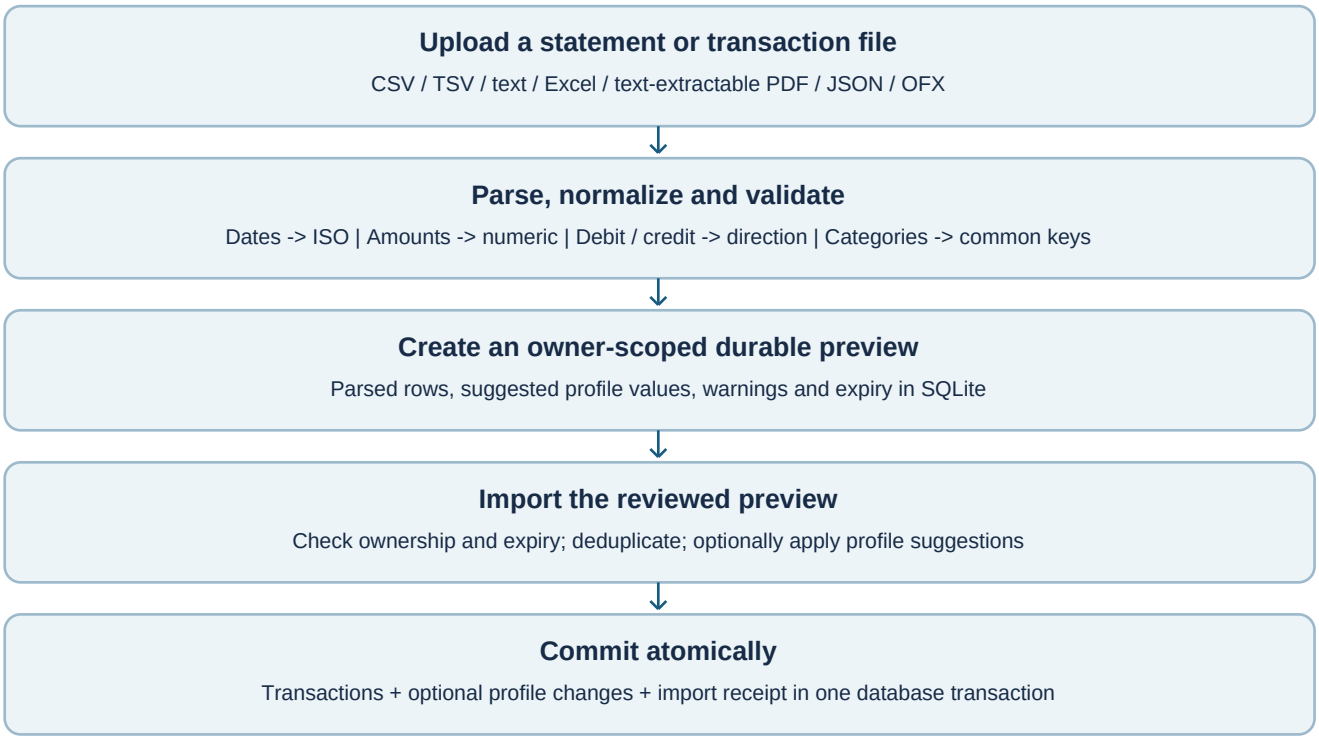
A user requests research on TCS and then asks, "What about its debt?" Conversation memory and company focus help resolve the reference. The system should use the resolved company and fetched statements rather than answering with an isolated generic definition of debt.

## Retry behavior

A successful repeated request_id can return the saved response. Reusing the id for a different message is an error. This is request-level replay, not a promise of exactly-once execution across every external side effect or deployment topology.

**Observability:** qa.answer logs channel, mode, route, correction, grounding, degradation and stage timings. total_ms captured in response metadata and the final log timer can differ because they are sampled at different points.

# Data ingestion and statement imports




## Canonical transaction contract

The parser emits date, merchant, positive amount, debit/credit direction and category. The storage mapping translates profile categories such as rent to housing and health to medical. Credits and expenses must remain distinct so income is not counted as spending.

## Why preview and atomic import matter

A preview exposes parsing problems before persistence. The import stores a receipt and reuses it on retries. SQLite writer serialization protects deduplication across overlapping previews and processes. A failed transaction should not leave only half the profile or transaction changes saved.

## Quality controls and limits

The parser bounds rows, spreadsheet expansion, archive members, sheet count, cell count and PDF processing. Unsupported or ambiguous inputs produce warnings or errors. Do not describe this as a general OCR pipeline: image-only scans need a separate extraction capability.

**Sources:** modules/finance/statement_import.py; shared/statement_store.py; shared/user_store.py; tests/finance/test_statement_durability.py.

# Personal-finance inference

**Inputs:** facts supplied in chat or the profile wizard, persisted transactions, income, age, dependents, EMI, assets, liabilities, goals and other known profile fields. Stored defaults are not necessarily user-confirmed facts; sources metadata tracks what is actually known.

## Two complementary computation paths

**Profile-aware deterministic engine:** computes net worth, financial-health components, goal funding, affordability, retirement/FIRE and what-if results from available facts. When a required fact is missing, the finance flow can ask a specific clarification.

**Model orchestration:** the structured finance orchestrator combines risk classification, spending forecasts, anomalies, constrained budgeting, tax computation and SIP allocation before asking the shared language-model client for narration.

| Component | Input and transformation | Output |
|---|---|---|
| XGBoost risk | Log income, age, dependents, debt/income, category ratios, volatility, channel shares and more | Risk class, confidence, SHAP factors |
| Spending LSTM | Monthly category spending / income; sequential context | Three-month category forecasts with P10/P50/P90 heads |
| Isolation Forest | Transaction-derived anomaly features | Unusual-transaction flags; not proof of fraud |
| Budget optimization | Income, fixed obligations, risk and constraints | Feasible category allocation or handled infeasibility |
| Deterministic tools | Explicit profile inputs and configured assumptions | SIP/EMI calculations, planning and tax outputs |

## Illustrative example, not a measured result

For monthly income Rs. 100,000 and EMI Rs. 15,000, debt-to-income is 0.15. This value is a feature, not the complete risk decision. The system should derive the remaining features from the user's actual history rather than inventing transactions.

**Sources:** modules/finance/pipeline.py; personal.py; engine.py; orchestrator.py; features.py; forecast_model.py.

# Offline training and model artifacts

| Pipeline | Data / preparation | Artifact or serving role |
|---|---|---|
| Risk training | Persona-conditioned synthetic household profiles and transactions; shared feature builder | XGBoost JSON and auxiliary artifacts |
| Spend training | Monthly category ratios and sequence windows; context versus future targets | PyTorch LSTM checkpoint |
| Tutor index build | Concept texts and metadata -> normalized Sentence-BERT embeddings | FAISS index.faiss + meta.json |
| Knowledge tracing | Generated concept / correctness sequences | LSTM knowledge-tracing checkpoint |
| Teaching policy | Training environment representing mastery and teaching decisions | DQN policy ZIP |

## Rebuild sequence from the repository

```
python -m scripts.generate_synthetic_transactions
python -m modules.finance.train.train_xgb_risk
python -m modules.finance.train.train_lstm_spend
python -m scripts.build_concept_kb
python -m scripts.generate_kt_sequences
python -m modules.tutor.train.train_lstm_kt
python -m modules.tutor.train.train_dqn_teacher
```

## How to defend the data choice

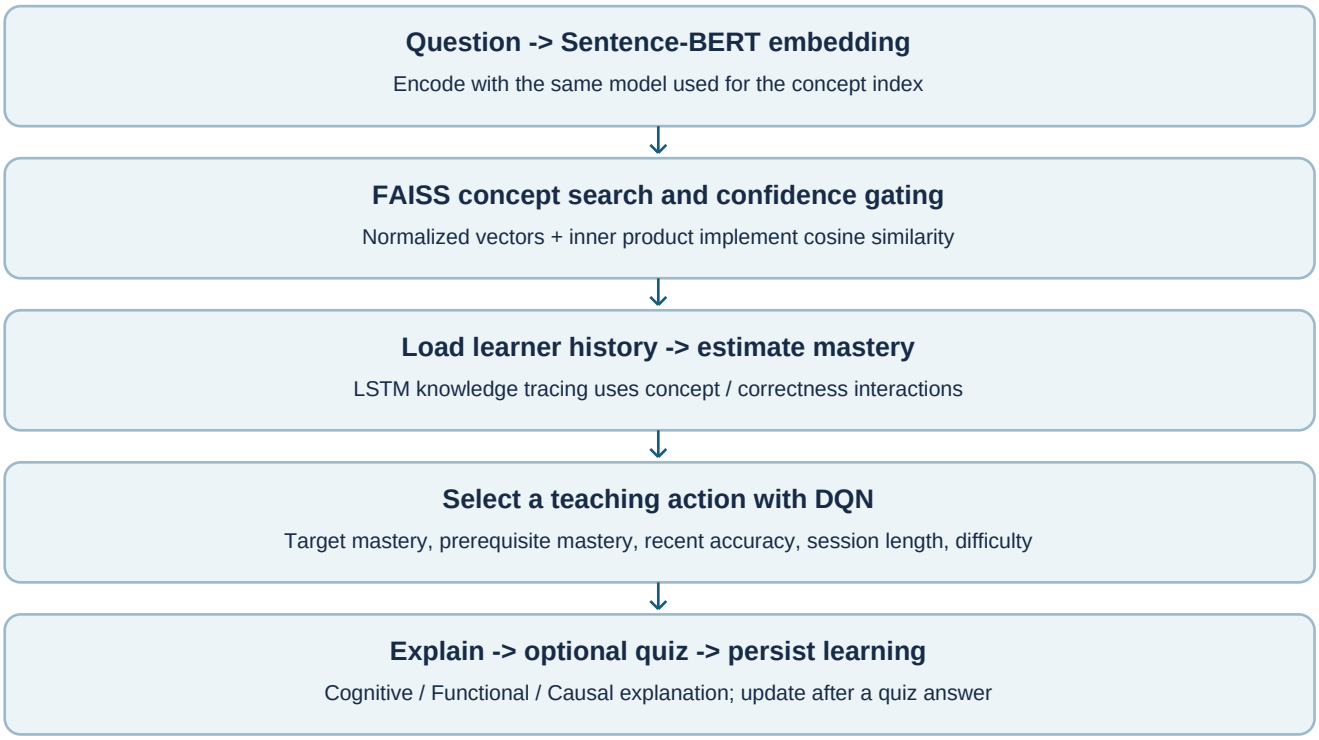
Synthetic data makes controlled experiments possible without collecting sensitive household histories. It also creates a substantial realism limit: a classifier can learn the generator's personas, and good synthetic performance does not establish real-world suitability. DQN rewards in a simulator do not establish improved student learning.

## Evaluation points to discuss

For a future stronger evaluation, hold out users for risk classification, use chronological splits for forecasting, fit preprocessing inside the training fold, and compare against simple baselines. Report macro-F1 / calibration for classification, MAE / quantile loss / interval coverage for forecasts, and independently measured learning gains for tutoring. These are evaluation proposals, not scores measured for this PDF.

**Artifacts:** models/finance/, models/tutor/, data/vector_store/. Model/data roots are configurable through config/paths.py.

# Adaptive tutor and semantic retrieval




## What makes this retrieval-based?

The retriever returns concept metadata from a curated finance knowledge base. It is semantic concept retrieval rather than unrestricted search over the internet. Low-confidence matches are rejected; the tutor pipeline can fall back to a conversational LLM response when no concept or local resource is available.

## Similarity equation

For normalized vectors q and d: similarity = q dot d = cosine(q, d). FAISS IndexFlatIP performs exact inner-product search over stored vectors. The current small concept collection does not require a distributed vector database.

## Adaptive behavior

The orchestration maps estimated mastery to explanation depth. The policy can choose a teaching action and request a quiz. Recording a quiz answer appends a correctness interaction, recomputes mastery and saves the learning state for later turns. A mastery probability is a model estimate, not a certified assessment of competence.

**Sources:** shared/vector_store.py; modules/tutor/retriever.py; orchestrator.py; knowledge.py; teaching.py; pipeline.py.

# Equity research: evidence to report

| Stage | Implementation behavior |
|---|---|
| Resolve | Map a company name / ticker to a target and identify the report type or entity. |
| Gather | Fetch statement tables, company and price data, headlines and available supporting information. Normalize units, periods and source differences. |
| Analyze | Compute statement trends, financial ratios, cash-flow measures, fundamental signals and investor-style lenses using code. |
| Value | Run a company-appropriate valuation: DCF for suitable companies; bank-specific dividend / book-value approaches; peer and other cross-checks. |
| Record provenance | A ledger distinguishes actuals, calculations, estimates, assumptions, source text and AI interpretation. |
| Audit numbers | Check evidence and consistency; valuation can be withheld when the audit finds blocking issues. |
| Narrate and debate | LLM agents write evidence-linked narrative and bull / bear / judge views; the implementation can run narrative tasks concurrently. |
| Audit narrative | Detect unsupported numeric claims and record verification findings. Missing source coverage stays visible. |
| Build and save | Create structured report data, summary and versioned owner-scoped saved reports; export through report renderers. |

## The valuation explanation to practise

A DCF forecasts free cash flow, discounts it and bridges enterprise value to equity value per share. WACC, terminal growth and other configured inputs are assumptions, not facts discovered by the LLM. Sensitivity tables and bull/base/bear cases expose how assumptions change the answer. A bank needs a different model because debt and capital have a different economic role.

## Important limits

A provenance ledger improves auditability; it cannot make an upstream statement correct. FinBERT sentiment is an optional local component. Evidence gaps, stale sources, currency mismatches or absent data must remain explicit. Persisting a report may fail separately from computing it; the code can still return an unsaved report.

**Sources:** modules/equity_research/intelligence/{pipeline,data,facts,analysis,valuation,audit,report}.py.

# Markets, news and forecast validation

## Market and news ingestion

Market analysis gathers real instrument / index context and uses it to ground the explanation. The repository integrates yfinance, screener.in for company statements, mfapi.in and AMFI for mutual-fund data, and RSS feeds for news. Provider coverage varies by workflow. Quote timestamps and market closures matter: "live" does not guarantee exchange-grade real-time data.

News is fetched, merged, deduplicated and ranked. Relevant headlines can be added to time-sensitive answers. Caching reduces repeated provider calls, but it creates a freshness tradeoff and must not hide a provider outage or stale snapshot.

## Forecast-validation data flow

• Retrieve and quality-check historical bars; record gaps, duplicate observations, outliers and corporate-action concerns; cache histories as Parquet where applicable.

• Construct historical features and forward log-return targets for configured horizons: 1, 5, 21, 63 and 126 bars.

• Run walk-forward evaluation with purging / embargo controls. Fit imputation, scaling and model selection only on allowed training observations.

• Compare candidates with baselines, select through validation and require confirmation on an untouched test segment. Permit "no_model_beats_baseline" and "insufficient_evidence" outcomes.

• Persist run metadata, progress and results. A FIFO worker dispatches jobs to spawned child processes so timeouts can terminate expensive work.

## Do not confuse two meanings of forecast

**Valuation scenarios** project operating assumptions into cash flows and fair value. **Statistical return forecasts** evaluate historical predictability against baselines. A plausible cash-flow story does not prove that the share price is predictable.

## Availability constraint

Automatic forecast and scenario jobs are skipped in account mode by the research pipeline, and integrations with unsafe shared global state are restricted. Do not present all optional compute features as universally enabled for signed-in users.

**Sources:** shared/news.py; modules/market/; modules/equity_research/forecast/{data,features,backtest,selection,models,jobs,contracts}.py; scenario_backtest/.

# Storage, identity and reliability

| State | Where / representation | Why it exists |
|---|---|---|
| Financial profile and transactions | SQLite via SQLAlchemy; profile facts and source metadata | Reuse known facts and support user edits |
| Conversation history and requests | Durable owner/session records plus a compatibility cache | Restart recovery, context and successful-request replay |
| Learning progress | SQLite mastery and interaction-history JSON | Personalized teaching across turns |
| Research reports | Versioned owner-scoped structured report JSON | Reopen, compare versions and export |
| Statement previews | SQLite rows, expiry and import results | Durable preview and atomic idempotent import |
| Semantic knowledge | FAISS files and metadata JSON | Concept retrieval |
| ML / historical artifacts | Local model checkpoints, datasets and Parquet caches | Inference and reproducible validation |

## Authentication versus identity

Private mode uses a browser/device owner id for local workflows; that id is not proof of a logged-in account. Account mode adds cookie-based authentication, password hashing and optional configured Google sign-in. The backend binds requests to the authenticated owner and checks ownership rather than trusting an arbitrary browser-supplied identifier.

## Concurrency and failure handling

In-process session locks serialize conversation work for a session. The database supports durable records. Statement imports also acquire a database writer transaction before deduplication. These protections have different scopes: an in-memory lock alone does not coordinate multiple backend processes.

The LLM client supports Groq with Gemini fallback. QA handling turns provider errors into controlled degraded responses. A number-matching guard can remove unsupported figures on supported evidence routes; finance engine output may be tagged rather than rewritten. Source interpretation, nonnumeric claims and all subtle contradictions remain outside any simple numeric check.

**Sources:** api/auth.py; shared/user_store.py; shared/conversation_memory.py; shared/statement_store.py; core/session.py; modules/qa/trust.py.

# Voice, avatars and latency

## Common flow

Speech recognition or a text message produces a query. The shared QA pipeline produces a written answer. Presentation code can create a shorter spoken version, synthesize audio and send it to a browser or an avatar integration. Mode and channel affect presentation; they should preserve the underlying answer and owner scope.

| Path | How to explain it |
|---|---|
| Text / conversational UI | Written answer; audio presentation can be prepared separately. |
| Live Avatar | Browser recognition, shared answer pipeline, spoken text and local Piper speech; browser face or 3D presentation. |
| Local avatar engine | Optional engine service with chunking, audio/video framing, motion and supported rendering backends. |
| LiveTalking | Optional separate lip-sync service and WebRTC presentation path. |
| Tavus | Optional hosted conversational-video integration; availability depends on configuration and account-mode restrictions. |

## A useful latency decomposition

End-to-end latency = recognition / input time + queue and warm-up time + domain data fetch + computation + LLM narration + speech synthesis + transport and browser buffering. Measure time to useful text, first audible word and sustained playback separately.

## Measured evidence and its limits

docs/MEASUREMENTS.md records partial measurements from 22 October 2026. Its browser avatar comparison uses a mocked chat reply and cached spoken text; it is not a complete chat-to-speech benchmark. It records GPU contention when both avatar services speak together, and some end-to-end measurements were still pending. Cite its setup and limitations if discussing those results.

## Failure and interruption

Optional rendering services can become unavailable while chat remains usable. Interrupting server generation does not automatically remove audio already buffered in a WebRTC client. Correct barge-in needs both cancellation upstream and management of client playback / queued media.

**Sources:** api/routes/live.py; modules/qa/present.py; ai/llm/spoken.py; ai/speech/live_tts.py; avatar_engine/; livetalking_adapter/; docs/MEASUREMENTS.md.

# Validation and a future scaling design

## What the repository tests

| Area | Examples present in tests/ |
|---|---|
| Shared answering | Unified pipeline, grounding, routing, intent, dialogue and adversarial behavior |
| Persistence and identity | Accounts, conversation memory, owner isolation, durable statement imports and audio persistence |
| Finance and tutoring | Financial tools, model wrappers, transaction/profile flows, retrieval, quizzes and mastery |
| Research and forecasts | Ratios, valuation, audit agents, report exports, historical backtests and leakage checks |
| Resource / media handling | Upload limits, compressed archives, avatar framing, replicas and stream behavior |

**Verification for this guide:** source inspection and PDF generation/structure checks only. No new test pass count, production load result or model performance score is claimed.

## What to measure before making performance claims

• Routing accuracy on independently labeled finance, tutor, market, equity and general questions.

• Model performance on held-out users or chronological future data, with baseline comparison and confidence intervals.

• Provider failure rate, cache age, p50/p95 request latency, active sessions, memory use and GPU contention.

• Grounding precision/recall, unsupported claims that slip through, and owner-isolation behavior across channels.

## Proposed production evolution: not implemented architecture

Keep the API stateless by moving process-local coordination and shared session state to suitable shared infrastructure. Move long-running forecasts and GPU rendering behind explicit worker queues with cancellation and resource limits. Use a shared transactional database and object storage for durable artifacts, version schemas and model files, and add provider health/freshness metrics.

Introduce distributed coordination only where measured concurrency requires it. Do not add Kafka, Spark, Airflow or a microservice fleet simply to make the diagram larger: the present system is an application pipeline with local jobs and provider fetching, not a distributed lakehouse or streaming ETL platform.

# Interview questions and short answers

**Why not send every question directly to an LLM?**
Profiles, calculations and valuations need structured inputs, reproducible arithmetic and persistent state. Domain tools compute results; the model helps explain them.

**Is ARIA a RAG application?**
The tutor retrieves semantically matched concepts with Sentence-BERT and FAISS. Other modules mainly use structured data and deterministic tools. The whole system is broader than one document-RAG pipeline.

**Why XGBoost and LSTM?**
XGBoost fits tabular risk features; the spend LSTM models temporal category patterns. That rationale still needs baseline evaluation: a complex model must demonstrate a useful improvement.

**What happens if a model or provider is missing?**
Behavior is route-specific: the tutor can use a conversational fallback, optional sentiment can be absent, and provider/LLM failures are surfaced through warnings or controlled degraded answers. Not every failure has an equivalent-quality fallback.

**How do you prevent hallucinations?**
Use fetched structured evidence, code-based calculations, provenance labels, audits and route-specific numeric checks. These reduce unsupported claims; they do not prove every sentence true.

**How do retries avoid duplicate data?**
Completed chat requests can replay by request_id. Statement imports separately use persisted previews, deduplication, writer serialization and an atomic import receipt.

**How do you avoid time-series leakage?**
Separate past training observations from future evaluation, purge overlapping targets, use an embargo and fit preprocessing inside each training split. Select using validation and confirm on untouched test data.

**What is the biggest limitation?**
Synthetic finance/training data and external-source quality limit real-world claims. Optional integrations and process-local state also constrain deployment scale.

**How would you scale it?**
First measure the bottleneck. Isolate heavy jobs, externalize shared state and coordination, strengthen database/artifact operations, and add observability before splitting services.

**What did you personally build?**
Name the exact modules you owned, trace one input to output, explain a difficult bug or tradeoff, and report only results you actually measured.

**Final rehearsal:** Draw the UI -> API -> QA pipeline -> four modules -> storage/providers diagram. Trace one uploaded statement and one research request. Then explain synthetic-data limits and one production improvement in your own words.

# Code map and revision checklist

| Responsibility | Files to open |
|---|---|
| Application / identity | api/main.py; api/auth.py; api/resource_limits.py; web/src/ |
| Unified question contract | modules/qa/models.py; pipeline.py; focus.py; trust.py; present.py |
| Routing / durable turns | core/router.py; core/orchestrator.py; core/session.py; shared/conversation_memory.py |
| Profile / import storage | shared/user_store.py; shared/statement_store.py; modules/finance/statement_import.py |
| Finance behavior | modules/finance/pipeline.py; personal.py; engine.py; orchestrator.py; features.py |
| Finance ML / optimization | modules/finance/risk_model.py; forecast_model.py; anomaly.py; budget_optimize.py; train/ |
| Tutor / semantic index | modules/tutor/orchestrator.py; retriever.py; knowledge.py; teaching.py; shared/vector_store.py |
| Market / news ingestion | modules/market/; shared/news.py; shared/company_resolver.py |
| Research / provenance | modules/equity_research/intelligence/{pipeline,data,facts,analysis,valuation,audit,report}.py |
| Historical validation | modules/equity_research/forecast/; modules/equity_research/scenario_backtest/ |
| LLM / voice / avatars | ai/llm/groq_client.py; ai/llm/spoken.py; ai/speech/; avatar_engine/; livetalking_adapter/ |
| Architecture / evidence | README.md; docs/QA_PIPELINE.md; docs/MEASUREMENTS.md; tests/ |

## Before joining the interview

- Be able to give the 90-second explanation without reading.
- Know which features are deterministic, learned, retrieved or generated.
- Have one real example of parsing, deduplication, retry handling or grounding ready.
- State whether a number is measured, configured, calculated or assumed.
- Describe your own contribution and one limitation candidly.

This document is an interview study guide based on local source inspection on 24 October 2026. Historical documentation and comments can lag implementation. Sources above are repository references rather than claims of live-provider availability.